

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTAD DE INGENIERÍA
Ingeniería de Sistemas

Project Definition Document

Data and analysis programming

Workshop #2

Students

Juan David Amaya Patiño 20221020057

Juan Pablo Mosquera 20221020026

Edward Jualian Garcia Gaitan 20212020136

Professor: Carlos Andrés Sierra Virguez

Date: April 2026

1 Executive Summary

Executive Summary

This report documents the technical architecture and pipeline implementation developed for the Public Opinion & Sentiment Analysis project, which focuses on collecting, processing, and analyzing public sentiment related to movies and streaming content. The system ingests data from two complementary sources: the **TMDb API** (The Movie Database), which provides structured metadata such as ratings, vote counts, popularity scores, and plot overviews; and Rotten Tomatoes, from which critic and audience scores are extracted via controlled web scraping of the publicly accessible `/browse/` section.

Technical Approach

The pipeline follows a medallion architecture organized into three layers: Bronze, Silver, and Gold. The Bronze layer stores raw data exactly as received from both sources, persisted as JSON files using a datetime-based naming convention

`source_YYYYMMDD_HHMMSS.json`. The Silver layer applies a comprehensive pre-processing pipeline that includes null handling, data type normalization, deduplication, outlier detection, and text normalization for downstream NLP tasks; processed outputs are stored in Parquet format (`topic_YYYYMMDD_HHMMSS.parquet`) for efficient columnar storage and compatibility with analytical frameworks. The Gold layer, planned for the following workshop, will aggregate Silver outputs into dashboard ready metrics including sentiment summaries, critic vs audience comparisons, and temporal trends.

Stack Decisions

The full stack runs inside a Docker Compose environment to ensure reproducibility and portability across development machines. Apache Airflow is used as the orchestration engine, leveraging the modern TaskFlow API for cleaner, more maintainable DAG definitions. Two DAGs are implemented: a Bronze ingestion DAG and a silver one. Pandas is selected as the primary data processing library for the Silver layer due to its mature ecosystem, straightforward API, and strong compatibility with Parquet via PyArrow. Parquet was chosen over CSV for Silver persistence because it provides schema enforcement, columnar compression, and native support for typed fields, which is essential for the PySpark-based Gold layer planned in Workshop #3.

Data extraction relies on the **requests** library for API calls and **BeautifulSoup4** for HTML parsing. **Plotly Dash** is proposed as the visualization layer for the final dashboard.

Pipeline Design

The end-to-end pipeline begins with scheduled extraction jobs that query selected TMDb endpoints (`/movie/top_rated`, `/movie/popular`) and scrape the Rotten Tomatoes browse page, persisting raw payloads in the Bronze layer. A **FileSensor**-triggered DAG detects new Bronze files and applies the preprocessing pipeline: duplicate removal, date normalization, score conversion from string percentages to numeric floats, null imputation, outlier handling via IQR-based filtering, and text cleaning of overview fields. The resulting Silver Parquet files retain a traceability link to their Bronze source through filename conventions. This modular design separates ingestion from transformation, simplifies debugging, and supports incremental improvements to the preprocessing logic in subsequent workshops.

2 Technical Architecture

2.1 Architecture Diagram

2.2 System Components Description

This section describes each component of the pipeline architecture, detailing its purpose, the technology employed, its inputs and outputs, and its interactions with adjacent components in the data flow.

2.2.1 Data Sources

The pipeline integrates two complementary external sources that together provide both structured metadata and public perception indicators for movies and streaming content.

TMDb API (The Movie Database). The primary structured source is the TMDb REST API, accessible at <https://api.themoviedb.org/3>. Authentication is handled via an API key passed as a query parameter in each request. The pipeline queries two main endpoints: `/movie/top_rated`, which returns a ranked list of top-rated films with metadata, and `/movie/popular`, which returns currently trending titles. Each response is a JSON object containing fields such as `id`, `title`,

`release_date`, `vote_average`, `vote_count`, `popularity`, `genre_ids`, and `overview`. The API does not enforce strict rate limits but recommends staying below approximately 40 requests per second. This source was selected for its reliability, consistent JSON structure, and rich metadata that serves as the structured backbone of the dataset.

Rotten Tomatoes (Web Scraping). The complementary source is the public browse page at https://www.rottentomatoes.com/browse/movies_in_theaters/sort:popular. Since Rotten Tomatoes does not offer a freely accessible public API, data is extracted via controlled web scraping using the `requests` and `BeautifulSoup4` libraries. Prior to implementation, the site's `robots.txt` was reviewed and confirmed that the `/browse/` path is not listed under any `Disallow` directive, ensuring compliance with the site's crawling policy. The scraper targets repeating movie card elements identified by `data-qa` attributes and extracts the following fields: `title`, `release_date`, `critics_score`, `audience_score`, `poster_url`, and `url`. This source contributes critic and audience perception indicators that complement the structural metadata provided by TMDb.

2.2.2 Orchestration Layer (Apache Airflow)

Pipeline orchestration is handled by **Apache Airflow 2.9.1**, running inside a Docker container as part of the project stack. Airflow manages the scheduling, execution, and monitoring of all pipeline tasks through Directed Acyclic Graphs (DAGs).

All DAGs are implemented using the modern **TaskFlow API**, which relies on the `@dag` and `@task` decorators to define pipelines in a clean, Pythonic style. This approach reduces boilerplate, improves readability, and aligns with current industry standards for Airflow development.

The orchestration layer consists of two DAGs. The first is the **Bronze Ingestion DAG**, which runs on a scheduled basis and is responsible for triggering extraction tasks for both data sources and persisting raw outputs in the Bronze layer. The second is the **Silver Processing DAG**, which is not time-scheduled but instead activated reactively through a `FileSensor` operator that monitors the `datalake_bronze/` directory for newly arrived files. Upon detection, it triggers the preprocessing pipeline and writes processed outputs to the Silver layer.

The Airflow environment is composed of four services running via Docker Compose: the **webserver**, which exposes the UI at `localhost:8080`; the **scheduler**, which manages DAG execution timing and task queuing; the **triggerer**, which is required for deferred operators such as `FileSensor`; and a **PostgreSQL 16** instance that serves as the Airflow metadata database. The `LocalExecutor` is used as the

execution backend, which is appropriate for the single-machine academic scope of this project.

2.2.3 Bronze Layer

The Bronze layer is the first persistence stage of the pipeline and stores raw data exactly as received from each extraction source, without applying any transformations. Its primary purpose is to preserve source integrity, ensure traceability, and support reprocessing in case errors are detected in downstream stages.

Each ingestion run produces one JSON file per source, persisted in the `datalake_bronze/` directory, which is mounted as a Docker volume to ensure data survives container restarts. Files are written using a datetime-based naming convention described in Section 2.4. No schema enforcement or field filtering is applied at this stage; the complete raw API response or scraping output is stored as-is.

The Bronze layer acts as the immutable historical record of all ingestion runs. It serves as the direct input to the Silver Processing DAG, which is triggered automatically upon the arrival of new files in this directory via the `FileSensor` operator.

2.2.4 Silver Layer

The Silver layer contains cleaned, normalized, and validated data produced by the Silver Processing DAG from Bronze source files. Its purpose is to transform raw, heterogeneous inputs into a consistent, analytically usable schema that can support both exploratory analysis and downstream aggregation.

The preprocessing pipeline applied at this stage includes the following operations: data type normalization (e.g., parsing date strings into `datetime` objects, converting percentage strings from Rotten Tomatoes into numeric floats), null and missing value handling according to field-specific strategies, deduplication of records across ingestion batches, outlier detection and handling in numeric fields using IQR-based filtering, and text normalization of the `overview` field for NLP readiness (lowercasing, punctuation removal, stop word filtering, and tokenization).

Processed outputs are persisted as **Parquet** files in the `datalake_silver/` directory using a timestamp-based naming convention. Parquet was selected over CSV for its columnar storage format, efficient compression, native support for typed schemas, and compatibility with both Pandas and PySpark, which is the processing framework planned for the Gold layer in Workshop #3. Each Silver file retains traceability to its Bronze source through filename conventions and optional metadata fields embedded in the output.

2.2.5 Gold Layer

The Gold layer is planned for implementation in Workshop #3 and is not yet operational in the current pipeline version. Its purpose is to aggregate Silver outputs into compact, business-oriented datasets designed for direct consumption by the analytical dashboard.

Planned Gold outputs include title-level sentiment summaries, comparative metrics between critic and audience scores, temporal sentiment trends, and ranked lists of highly positive or negative content. These datasets will be produced using **PySpark**, which can load multiple Silver Parquet files across different time ranges simultaneously, leveraging the timestamp-based filename convention established in the Silver layer. The Gold outputs will serve as the data source for a **Plotly Dash** dashboard targeted at Brand Analysts and Content Analysts.

2.3 Technology Stack Justification

Table 1 summarizes the technologies used at each stage of the pipeline and the rationale behind each selection.

Table 1: Technology stack and justification by pipeline component

Component	Technology	Justification
API Extraction	Python requests	Lightweight HTTP client with native JSON parsing; sufficient for REST API consumption without overhead of heavier frameworks.
Web Scraping	BeautifulSoup4	Mature HTML parsing library with flexible selector support; appropriate for the static card-based structure of the Rotten Tomatoes browse page.
Orchestration	Apache Airflow 2.9.1	Industry-standard workflow orchestration tool with native support for DAGs, scheduling, sensors, and monitoring UI. TaskFlow API reduces boilerplate.
Containerization	Docker Compose	Ensures full environment reproducibility across machines; isolates services and manages inter-service networking and volume mounts.
Silver Processing	Pandas + PyArrow	Pandas provides a mature, well-documented API for data cleaning and transformation at the dataset sizes handled in this project. PyArrow enables efficient Parquet read/write with schema enforcement.
Silver Storage	Parquet	Columnar format with compression and typed schema support; compatible with both Pandas (Silver) and PySpark (Gold), enabling seamless layer transitions.
Metadata Database	PostgreSQL 16	Required by Airflow for storing DAG state, task instances, and run history. Selected for its stability and native support in the official Airflow image.
Dashboard (planned)	Plotly Dash	Python-native interactive visualization framework; integrates directly with Pandas DataFrames and requires no separate frontend stack.

2.4 File Nomenclature Convention

A consistent file naming convention is enforced across both the Bronze and Silver layers to ensure traceability, support automated downstream detection, and facilitate time-based loading in the Gold layer.

Bronze layer files follow the pattern:

`source_YYYYMMDD_HHMMSS.json`

where `source` identifies the origin of the data (e.g., `tmdb_top_rated`, `tmdb_popular`, `rottentomatoes`), and the datetime suffix reflects the exact moment of ingestion. Each execution of the Bronze DAG produces one file per source, ensuring that separate runs are never overwritten. Example filenames:

- `tmdb_top_rated_20260410_143022.json`
- `rottentomatoes_20260410_143055.json`

Silver layer files follow the pattern:

`topic_YYYYMMDD_HHMMSS.parquet`

where `topic` reflects the analytical subject of the processed dataset (e.g., `movies_sentiment`), and the datetime suffix corresponds to the processing run timestamp, not the ingestion timestamp. This distinction is intentional: it allows the Gold layer in Workshop #3 to load all Silver files within a given time range using PySpark's wildcard path resolution. Example filename:

- `movies_sentiment_20260410_143512.parquet`

This convention ensures full traceability from Silver outputs back to their Bronze sources by comparing timestamps, and provides a natural partitioning strategy for future incremental processing.

3 Preprocessing Pipeline Definition

The Silver layer preprocessing pipeline transforms raw Bronze JSON files into cleaned, schema-normalized Parquet datasets suitable for downstream analytics. Rather than collapsing all records into a single mixed dataset, the preprocessing logic produces three separate Silver outputs: `silver_titles`, `silver_rt_reviews`, and `silver_tmdb_reviews`. This design improves readability, preserves analytical granularity, and simplifies future aggregation in the Gold layer. The preprocessing decisions documented below are grounded in the Bronze samples collected during Workshop #1 and in the structures observed during Workshop #2 implementation. [?]

3.1 Null Handling

The preprocessing pipeline applies field-specific strategies for missing and null values. Empty strings are first normalized to `null` so that they can be handled consistently across sources. For title-level datasets, rows are only dropped when critical identifiers are absent, such as a missing canonical title. Optional fields such as `poster_url`, `overview`, `critics_score`, and `audience_score` are preserved as null when not available, since these missing values may reflect genuine source limitations rather than data quality errors.

In the review datasets, raw review text is considered essential for downstream NLP tasks. Therefore, review rows are retained only if a valid title reference and non-empty review text are present. Fields such as `publication`, `original_score`, or `rating` are optional and remain null when not provided by the source. Boolean attributes such as `is_verified` and `is_top_critic` are normalized whenever possible and otherwise stored as null.

This conservative null-handling strategy was chosen because the project prioritizes traceability and preservation of meaningful records over aggressive dropping or statistical imputation. Since the Silver layer is still intended to remain close to the raw evidence, missing values are generally flagged and preserved rather than replaced with synthetic estimates. [?]

3.2 Data Type Normalization

All Silver datasets enforce a normalized schema. Column names are converted to `snake_case`, string fields are stripped of surrounding whitespace, and numeric and temporal values are converted into appropriate types.

For Rotten Tomatoes, percentage strings such as "88%" are converted to numeric values (`float`) in the fields `critics_score` and `audience_score`. Release date strings such as "Opened Apr 01, 2026" or "Streaming Apr 1, 2026" are parsed into ISO-style dates, from which `release_year` is also derived. For TMDb, native ISO dates and timestamps are parsed directly into normalized date fields. Review dates from both TMDb and Rotten Tomatoes are stored in normalized temporal format to facilitate future time-based aggregation.

Boolean fields are standardized into proper `True/False` values when possible. The field `genre_ids` is serialized consistently as a JSON string rather than preserved as a nested Python list, which ensures a stable Parquet schema and simpler interoperability with downstream tools such as Pandas and PySpark. This was considered a practical trade-off for Workshop #2, where portability and schema stability are more important than preserving nested list semantics at this layer.

3.3 Text Preprocessing for NLP

The Silver layer preserves the original review text in the field `review_text`, while also generating a lightly normalized field `review_text_clean`. This design ensures that downstream NLP workflows can choose between raw and normalized text representations.

The cleaning process applied in Silver is intentionally light:

- lowercasing,
- HTML entity decoding where necessary,
- whitespace normalization,
- basic punctuation simplification,
- removal of repeated spacing and line breaks.

More aggressive NLP preprocessing steps such as stop word removal, stemming, lemmatization, or token-level filtering are intentionally postponed to later stages of the project. The Silver layer should prepare text for analysis, not finalize it. This separation keeps Silver close to source fidelity while still enabling future sentiment analysis and text mining in the Gold or notebook exploration stages.

3.4 Outlier Detection and Handling

Outlier handling is defined at the design level, although it is applied conservatively in the first Silver implementation. Numeric fields such as `popularity`, `vote_average`, `vote_count`, `critics_score`, and `audience_score` are the main candidates for outlier analysis.

The preferred statistical strategy is the Interquartile Range (IQR) method, since it is more robust than Z-score filtering for skewed distributions and small-to-medium datasets. For example, fields like `vote_count` or `popularity` may naturally contain extreme but valid values for blockbuster titles, so outlier handling must not blindly remove high-impact records. In this first version of the Silver pipeline, outlier detection is therefore treated primarily as a flagging and observational step rather than aggressive row exclusion.

This decision is justified by the academic scope of Workshop #2: the priority is to produce valid, traceable Silver datasets, while more refined distribution-sensitive treatment can be incorporated in Workshop #3 once broader historical coverage is available.

3.5 Deduplication

Deduplication is performed using business-key logic specific to each Silver output:

- **silver_titles:** deduplicated using `source + source_title + url + source_id`
- **silver_rt_reviews:** deduplicated using `canonical_title + review_type + reviewer_name + review_date + review_text`
- **silver_tmdb_reviews:** deduplicated using `movie_id + review_id`

This approach was selected because exact row equality is too strict across ingestion batches, while these business keys capture the identity of each logical entity more reliably. Deduplication is especially important in a pipeline scheduled periodically, where the same title or review may appear in multiple Bronze snapshots. The goal is to keep Silver analytically clean while still preserving the temporal lineage of each run.

3.6 Schema Enforcement

Each Silver family enforces a target schema before Parquet persistence. Missing columns are added as null to ensure stable structure across runs, and output files are written only after the datasets conform to the expected Silver schema.

The three enforced output families are:

- **silver_titles:** unified title-level metadata combining TMDb catalog snapshots and Rotten Tomatoes catalog records.
- **silver_rt_reviews:** flattened critic and audience reviews from Rotten Tomatoes.
- **silver_tmdb_reviews:** flattened movie review records from TMDb.

All Silver outputs include lineage fields such as `bronze_file_name`, `bronze_source_type`, and `processed_at`, enabling traceability from processed Parquet files back to the originating Bronze inputs.

3.7 Justification Based on Data

The preprocessing strategy is directly motivated by the Bronze data observed in Workshops #1 and #2. TMDb catalog files contain structured movie records with consistent numeric and temporal fields, whereas Rotten Tomatoes catalog files contain score strings and semi-structured release date text that require parsing and conversion. Review files from Rotten Tomatoes contain nested critic and audience review arrays, while TMDb review files contain nested `movie` and `reviews` objects. These structural differences justify the use of separate Silver families and tailored normalization logic for each one.

The decision to preserve raw review text while adding a lightly normalized companion field is also grounded in the intended final use case: public sentiment analysis for entertainment content. Since review text is the core input for later NLP tasks, it must remain intact while also being made easier to process in future stages. Overall, the preprocessing design balances analytical readiness, schema consistency, and source fidelity, which is exactly the role expected from the Silver layer in the medalion architecture.

4 Docker Stack Description

4.1 Services

4.1.1 Airflow Webserver

The Airflow Webserver is containerized as part of a Docker Compose stack using the official Apache Airflow image (version 2.9.1), built from the provided `Dockerfile`. It inherits common configuration via the `&airflow-common` anchor for consistency across services.

Service Configuration (`docker-compose.yaml`):

- **Image:** Built from `Dockerfile`, which extends `apache/airflow:2.9.1` with custom dependencies defined in `requirements.txt` (e.g., `requests`, `pandas`, `beautifulsoup4`).
- **Environment:**
 - `PYTHONPATH`: Includes `/opt/airflow/dags` and `/opt/airflow` to enable importing custom modules such as `utils.tmdb_client`.

- Inherits database credentials (e.g., `POSTGRES_USER`, `POSTGRES_PASSWORD`) and Airflow settings from `.env`.
- **Volumes:**
 - Mounts `datalake_gold` for shared data access to processed outputs.
 - Implicitly mounts DAGs, logs, and plugins from the project root.
- **User:** Runs as `AIRFLOW_UID:50000` for security.
- **Dependencies:** Relies on `postgres` for metadata storage.
- **Ports:** Exposed on port 8080 (default) for web access.
- **Networks:** Uses an isolated network for inter-service communication.

Role in Stack: The webserver provides the Airflow UI at `http://localhost:8080`, enabling DAG management, task monitoring, and manual triggering of pipeline runs. It integrates with the scheduler for real-time updates and with the PostgreSQL database for state persistence. Authentication is handled via environment variables `_AIRFLOW_WWW_USER_USERNAME` and `_AIRFLOW_WWW_USER_PASSWORD`.

To start the service, run `docker compose up webserver`, and inspect its logs with `docker compose logs webserver`. For production deployments, it is recommended to add health checks, SSL termination, or a reverse proxy in front of the webserver.

4.1.2 Airflow Scheduler

The Airflow Scheduler is containerized using the same base image and common configuration as the webserver, ensuring consistency across all Airflow services. It is responsible for DAG parsing, task scheduling, and execution coordination.

Service Configuration (docker-compose.yml):

- **Image:** Built from Dockerfile (extends `apache/airflow:2.9.1` with dependencies such as `requests` and `pandas`).
- **Environment:**
 - `PYTHONPATH`: Includes `/opt/airflow/dags` and `/opt/airflow` for importing custom modules.

- Inherits database and Airflow settings from `.env` (e.g., executor type, configured as `LocalExecutor` in this project).
- **Volumes:**
 - Mounts `datalake_gold` for data access.
 - Shares DAGs, logs, and plugins with other services.
- **User:** Runs as `AIRFLOW_UID:50000`.
- **Dependencies:** Depends on `postgres` for metadata and task state persistence.
- **Command:** Runs `airflow scheduler` to process DAG files and queue tasks.
- **Networks:** Isolated network for communication with the webserver and the database.

Role in Stack: The scheduler parses DAG files (e.g., `bronze_ingestion_dag.py`) and schedules tasks according to their defined intervals (e.g., daily runs identifiable via run IDs such as `scheduled__2026-04-13T00:00:00+00:00`). It executes tasks via `LocalExecutor`, handling retries and inter-task dependencies. The scheduler integrates with the webserver for UI status updates and writes execution logs under `logs/scheduler/`.

To start the service, run `docker compose up scheduler` and monitor it with `docker compose logs scheduler`. For scaling to multi-worker setups, switch to `CeleryExecutor` in production and ensure `.env` is correctly configured. If DAGs fail to parse, verify `PYTHONPATH` settings and all imports inside the `utils/` package.

4.1.3 Airflow Triggerer

The Airflow Triggerer is containerized as part of the Docker Compose stack, using the same base image and common configuration as the other Airflow services. It is responsible for managing deferred tasks that rely on external events or asynchronous operations.

Service Configuration (`docker-compose.yaml`):

- **Image:** Built from Dockerfile (extends `apache/airflow:2.9.1` with project dependencies).

- **Environment:**
 - PYTHONPATH: Includes `/opt/airflow/dags` and `/opt/airflow` for custom modules.
 - Inherits database and Airflow settings from `.env`.
- **Volumes:**
 - Mounts `datalake_gold` for data access.
 - Shares DAGs, logs, and plugins with other services.
- **User:** Runs as `AIRFLOW_UID:50000`.
- **Dependencies:** Depends on `postgres` for deferred task state management.
- **Command:** Runs `airflow triggerer` to process deferred tasks.
- **Networks:** Isolated network for integration with the scheduler and webserver.

Role in Stack: The triggerer handles tasks that are paused and resumed based on external events — for example, API responses or file availability — improving efficiency for long-running or I/O-bound operations. It complements the scheduler by offloading non-immediate task execution, and its behavior is observable in DAG logs associated with tasks such as `extract_tmdb_trending`. In this project, the triggerer is essential for the `FileSensor`-based trigger of the Silver Processing DAG, which monitors the `datalake_bronze/` directory for newly ingested files.

The triggerer is not directly visible in the Airflow UI but silently supports complex workflows defined in `bronze_ingestion_dag.py` and `silver_processing_dag.py`. To start it, run `docker compose up triggerer` and monitor with `docker compose logs triggerer`. This component is optional for simple pipelines but required for any DAG using deferrable operators. It is supported from Airflow version 2.2 onwards.

4.1.4 PostgreSQL Database

PostgreSQL serves as the metadata database for Apache Airflow, storing DAG run history, task instances, connections, variables, and execution state. It is containerized within the Docker Compose stack and configured with a named volume to ensure data persistence across container restarts.

Service Configuration (docker-compose.yaml):

- **Image:** postgres:16, consistent with the Airflow metadata requirements for this project.
- **Environment:**
 - POSTGRES_USER: airflow
 - POSTGRES_PASSWORD: airflow
 - POSTGRES_DB: airflow
 - These values are loaded from .env for security.
- **Volumes:**
 - postgres-db-volume: A named Docker volume that persists the database contents across container restarts and re-deployments.
- **Ports:** Exposed on port 5432 (default) for external access if required.
- **Networks:** Isolated network for secure communication with all Airflow services (webserver, scheduler, and triggerer).

Role in Stack: PostgreSQL acts as the backend for Airflow’s internal state management, enabling reliable tracking of DAG runs such as those produced by `bronze_ingestion_dag.py` and `silver_processing_dag.py`. It stores task statuses, execution history, and run logs accessible through the `logs/` directories. It also supports the broader data ingestion workflow by providing a stable metadata store for the orchestration engine.

PostgreSQL starts automatically when executing `docker compose up`. If direct database access is needed, it can be reached via:

```
docker compose exec postgres psql -U airflow -d airflow
```

To create a backup of the metadata database, use:

```
docker compose exec postgres pg_dump -U airflow airflow > backup.sql
```

The named volume must not be deleted between sessions to preserve pipeline history. If connection issues arise, verify the `.env` credentials and confirm that the `postgres` service is healthy before starting the Airflow services.

4.2 Volumes and Persistence

All data produced inside Docker containers is persisted to the host filesystem through volume mounts defined in the `x-airflow-common` block of the `docker-compose.yaml`. This ensures that pipeline outputs survive container restarts and `docker compose down` operations, as long as the `-v` flag is not used.

Table 2 describes each mounted volume and its purpose.

Table 2: Docker volume mounts and their purpose

Host Path	Container Path	Purpose
<code>airflow/dags/</code>	<code>/opt/airflow/dags</code>	DAG files with live reload
<code>airflow/utils/</code>	<code>/opt/airflow/utils</code>	Shared Python modules
<code>airflow/logs/</code>	<code>/opt/airflow/logs</code>	Task execution logs
<code>airflow/plugins/</code>	<code>/opt/airflow/plugins</code>	Custom Airflow plugins
<code>airflow/config/</code>	<code>/opt/airflow/config</code>	Airflow configuration files
<code>datalake_bronze/</code>	<code>/opt/airflow/datalake_bronze</code>	Bronze JSON ingestion outputs
<code>datalake_silver/</code>	<code>/opt/airflow/datalake_silver</code>	Processed Parquet outputs
<code>datalake_gold/</code>	<code>/opt/airflow/datalake_gold</code>	Aggregated analytics (planned)

The PostgreSQL metadata database is persisted through a named Docker volume `postgres-db-volume`, meaning that DAG state, task history, and run records are retained across restarts without requiring re-initialization.

4.3 Dependencies

Dependencies are managed through two files: `requirements.txt`, which lists all Python packages, and a custom `Dockerfile`, which installs them on top of the official Airflow base image.

The `Dockerfile` extends `apache/airflow:2.9.1` and installs all packages at build time, which is the recommended approach over runtime installation via `_PIP_ADDITIONAL_REQUIREMENTS`.

for production-like setups. The `docker-compose.yaml` references it via `build: .` instead of a pre-built image.

Table 3 summarizes the key dependencies and their role in the pipeline.

Table 3: Key Python dependencies by pipeline role

Package	Version	Role
<code>requests</code>	2.31.0	HTTP client for TMDb API calls
<code>beautifulsoup4</code>	4.12.2	HTML parsing for Rotten Tomatoes scraping
<code>lxml</code>	4.9.3	High-performance HTML parser backend for BeautifulSoup
<code>pandas</code>	2.1.4	Data cleaning and transformation in Silver layer
<code>pyarrow</code>	14.0.1	Parquet read/write with schema enforcement
<code>numpy</code>	1.26.2	Numerical operations for outlier detection
<code>python-dotenv</code>	1.0.0	Loading environment variables from <code>.env</code> file
<code>jupyter</code>	1.0.0	Local exploration of Silver Parquet files
<code>apache-airflow-providers-sqlalchemy</code>	5.7.0	PostgreSQL connection support for Airflow

4.4 Setup Instructions

The following steps describe how to build and run the full stack locally from a clean clone of the repository.

Prerequisites: Docker Desktop (or Docker Engine + Compose plugin) must be installed and running. The project was tested on Docker Compose v2.

1. **Create the `.env` file** in the project root with the required environment variables:

```
AIRFLOW_UID=50000
TMDB_ACCESS_TOKEN=your_tmdb_bearer_token
TMDB_API_KEY=your_tmdb_api_key
FERNET_KEY=your_fernet_key
```

```
_AIRFLOW_WWW_USER_USERNAME=airflow
_AIRFLOW_WWW_USER_PASSWORD=airflow
```

A Fernet key can be generated with:

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().d
```

2. **Build the custom Docker image** defined in the `Dockerfile`, which installs all dependencies from `requirements.txt`:

```
docker compose build
```

3. **Initialize the Airflow metadata database** and create the default admin user. This step is only required once:

```
docker compose up airflow-init
```

Wait until the service exits with code 0 before proceeding.

4. **Start all services** in detached mode:

```
docker compose up -d
```

This launches the webserver, scheduler, triggerer, and PostgreSQL services.

5. **Verify that all containers are healthy:**

```
docker compose ps
```

All services should show **healthy** or **running** status within approximately 60 seconds.

6. **Access the Airflow UI** at `http://localhost:8080` using the credentials defined in the `.env` file (default: `airflow / airflow`).

7. To stop the stack without losing data:

```
docker compose down
```

To stop and remove all volumes for full reset:

```
docker compose down -v
```

4.5 Airflow UI Verification

After completing the setup process, the Airflow web interface is accessible at <http://localhost:8080>. Figure 2 shows the DAG list view confirming that the `bronze_ingestion` DAG is registered, active, and has completed at least one successful run.

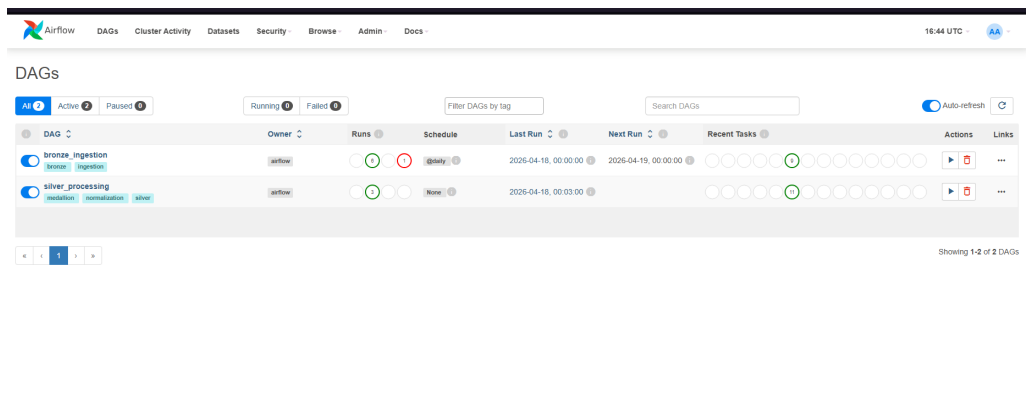


Figure 1: Airflow UI at `localhost:8080` showing the bronze ingestion DAG and silver.

5 DAG Documentation

5.1 Bronze Ingestion DAG

5.1.1 Description

The Bronze Ingestion DAG (`bronze_ingestion`) is responsible for extracting raw data from both external sources — TMDb API and Rotten Tomatoes — and persisting the results as timestamped JSON files in the `datalake_bronze/` directory.

It constitutes the entry point of the pipeline and operates without applying any transformation to the extracted data.

5.1.2 Tasks

The DAG is implemented using the TaskFlow API and defines eight tasks organized in two parallel branches that converge in a final validation step, as shown in Figure ??.

TMDb branch:

- **extract_tmdb_trending:** Queries `/trending/movie/day` and persists the daily trending snapshot.
- **extract_tmdb_now_playing:** Queries `/movie/now_playing` across multiple pages (configurable via `TMDB_NOW_PLAYING_PAGES`, default 3) to retrieve movies currently in theaters.
- **extract_tmdb_reviews:** Receives the output file paths from the two previous tasks, merges their movie IDs (deduplicating via `set()`), and fetches user reviews from `/movie/{id}/reviews` for up to `TMDB_REVIEW_MOVIES` unique titles.

Rotten Tomatoes branch:

- **extract_rt_theaters:** Scrapes `/browse/movies_in_theaters` and extracts title, release date, critics score, audience score, poster URL, and movie URL for all listed titles.
- **extract_rt_at_home:** Scrapes `/browse/movies_at_home` with the same field structure, capturing streaming and VOD content.
- **extract_rt_reviews:** Re-scrapes the theaters listing and, for up to `RT_MAX_MOVIES` titles (default 5), fetches both critic and audience reviews using RT's internal `napi/rtcf` JSON endpoint, discovered dynamically from each movie's review page metadata.
- **extract_rt_at_home_reviews:** Equivalent to the previous task but applied to the at-home listing.

Validation:

- **validate_files:** Receives the file paths produced by all six extraction tasks and verifies that each file exists on disk and has a non-zero size. The DAG run fails explicitly if any output is missing or empty.

5.1.3 Schedule

The DAG is scheduled with `@daily` and `catchup=False`, meaning it executes once per day at midnight UTC without backfilling historical runs. This cadence is appropriate because both TMDb trending data and Rotten Tomatoes scores are updated daily, and accumulating one snapshot per day is sufficient to analyze temporal evolution of popularity and sentiment over time. Key parameters are configurable via environment variables (`TMDB_NOW_PLAYING_PAGES`, `TMDB_REVIEW_MOVIES`, `RT_MAX_MOVIES`, etc.) without modifying the DAG code.

5.1.4 Outputs

Each successful run produces seven JSON files in `datalake_bronze/`, following the `source_YYYYMMDD_HHMMSS.json` convention:

Table 4: Bronze output files per DAG run

File	Content
<code>tmdb_trending_movie_day_*.json</code>	Daily trending snapshot from TMDb
<code>tmdb_now_playing_*.json</code>	Movies currently in theaters (multi-page)
<code>tmdb_movie_reviews_*.json</code>	User reviews for deduplicated movie pool
<code>rottentomatoes_theaters_*.json</code>	Scores for movies in theaters
<code>rottentomatoes_at_home_*.json</code>	Scores for streaming/VOD titles
<code>rottentomatoes_reviews_*.json</code>	Critic and audience reviews — theaters
<code>rottentomatoes_at_home_reviews_*.json</code>	Critic and audience reviews — at home

5.2 Silver Processing DAG

5.2.1 Description

The Silver Processing DAG (`silver_processing`) is responsible for detecting new Bronze JSON files, applying the preprocessing logic defined in the previous section, and persisting normalized outputs as Parquet files in the `datalake_silver/` directory. Unlike the Bronze ingestion DAG, whose role is pure extraction, the Silver

DAG performs the first substantial transformation stage of the pipeline and acts as the bridge between raw ingestion and analytical preparation.

The DAG is implemented using the Airflow TaskFlow API and processes Bronze files in a source-aware manner. Instead of collapsing all records into a single mixed dataset, it routes Bronze inputs into three independent Silver outputs: `silver_titles`, `silver_rt_reviews`, and `silver_tmdb_reviews`. This separation improves maintainability, preserves data semantics, and makes downstream Gold-layer aggregation more straightforward.

5.2.2 FileSensor Trigger

The Silver DAG is triggered reactively through a file-watching strategy equivalent to a `FileSensor`. Its role is to monitor the `datalake_bronze/` directory and wait until JSON files are available for processing. This design decouples ingestion from transformation: Bronze can run on its own schedule, and Silver begins only when source files are actually present.

The sensor logic is configured with a `poke_interval` and `timeout` appropriate for the local academic environment. Once Bronze files are detected, the DAG proceeds to list and classify them according to their filename pattern. This behavior aligns with the Workshop #2 requirement that Silver be triggered automatically upon the arrival of new Bronze files rather than through a fixed time schedule. [?]

5.2.3 Tasks

The Silver DAG is organized into the following functional tasks:

- `wait_for_bronze_files`: waits until at least one JSON file is detected in `datalake_bronze/`.
- `list_files`: scans the Bronze directory and returns the available JSON file paths for processing.
- `classify_files`: identifies each Bronze file type from its filename prefix and groups it into one of the three Silver families: `silver_titles`, `silver_rt_reviews`, or `silver_tmdb_reviews`. Files that do not match the expected patterns are logged and ignored.
- `get_run_stamp`: derives a deterministic timestamp from the Airflow logical date. This value is later used in output file naming to support idempotent behavior across retries of the same DAG run.

- **process_titles**: consumes title-level Bronze files from TMDb catalog snapshots and Rotten Tomatoes catalog scrapes. It applies normalization, date parsing, percentage conversion, schema alignment, and deduplication before writing the resulting **silver_titles** Parquet file.
- **process_rt_reviews**: consumes Rotten Tomatoes review payloads, flattens critic and audience review arrays, preserves the raw review text, creates **review_text_clean**, normalizes scores and dates, and writes the resulting **silver_rt_reviews** Parquet file.
- **process_tmdb_reviews**: consumes TMDb review payloads, flattens nested review objects under each movie, retains movie-level metadata for context, normalizes review dates and ratings, and writes the resulting **silver_tmdb_reviews** Parquet file.
- **summarize_outputs**: logs the final Silver file paths generated by the DAG and provides a compact summary of the run outputs.

Each processing task includes basic logging and exception handling. This design makes the DAG easier to debug and ensures that failures can be isolated to one output family without obscuring the broader pipeline behavior.

5.2.4 Outputs

Each successful Silver DAG run may produce up to three Parquet files in **datalake_silver/**, depending on which Bronze file families are available. The outputs follow a timestamp-based naming convention:

```
silver_titles_YYYYMMDD_HHMMSS.parquet
silver_rt_reviews_YYYYMMDD_HHMMSS.parquet
silver_tmdb_reviews_YYYYMMDD_HHMMSS.parquet
```

These filenames are derived from the Airflow logical run timestamp, which makes them stable for retries of the same run and therefore supports practical idempotency. The files are written atomically through a temporary-file replacement strategy, reducing the risk of partially written outputs if a task is interrupted or retried.

The three output families are summarized below:

Table 5: Silver output files and analytical role

Output file	Purpose
<code>silver_titles_*.parquet</code>	Unified title-level dataset combining TMDb catalog metadata and Rotten Tomatoes catalog indicators.
<code>silver_rt_reviews_*.parquet</code>	Flattened Rotten Tomatoes critic and audience reviews with review text, source metadata, and score context.
<code>silver_tmdb_reviews_*.parquet</code>	Flattened TMDb review records enriched with movie-level metadata for later sentiment and comparative analysis.

5.3 Traceability Strategy

Traceability between Bronze and Silver is preserved through both naming conventions and explicit lineage fields. Every Silver record includes metadata columns such as `bronze_file_name`, `bronze_source_type`, and `processed_at`, which allow analysts and developers to identify the exact Bronze snapshot from which the processed row was derived.

At the file level, the Silver output naming convention preserves the processing run timestamp, while the Bronze files retain the ingestion timestamp. Together, these two timestamps make it possible to reconstruct the relationship between extraction and processing events across pipeline runs. This design supports reproducibility, simplifies debugging, and prepares the project for Workshop #3, where multiple Silver files from different processing windows will be loaded simultaneously into the Gold layer. [?]

6 Results Evidence

6.1 Airflow UI Execution

Figure 2 shows the Airflow web interface captured during an active pipeline session. Both DAGs are enabled and reflect multiple completed runs, confirming that the end-to-end orchestration is fully operational.

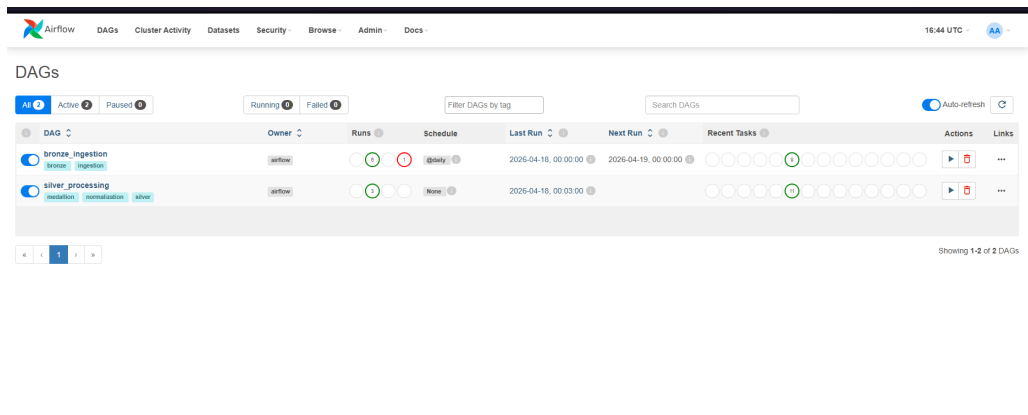


Figure 2: Airflow UI showing execution state of `bronze_ingestion` and `silver_processing` DAGs as of 2026-04-19.

Table 6: DAG execution summary observed in the Airflow UI.

DAG ID	Schedule	Successful	Failed	Last Run	Next Run	Status
<code>bronze_ingestion</code>	@daily	8	1	2026-04-19 16:53	2026-04-19 00:00	Active
<code>silver_processing</code>	FileSensor	4	1	2026-04-19 16:56	—	Active

The `bronze_ingestion` DAG completed 8 of 9 runs successfully. The single failure corresponds to a transient HTTP error during the Rotten Tomatoes scraping task, which is expected given the inherent fragility of web scraping against structural site changes. The most recent run executed all 9 tasks successfully, as indicated by the green circle with count 9 in the Recent Tasks column.

The `silver_processing` DAG carries no fixed schedule. It is triggered exclusively by the `FileSensor` operator upon detection of new JSON files in `datalake_bronze/`. Four successful runs are recorded, with the DAG activating approximately three minutes after each Bronze run, confirming that the sensor-based trigger mechanism is functioning correctly.

6.2 Bronze Layer Output

The Bronze layer stores raw ingestion outputs exactly as received from each source, with no transformation beyond JSON serialisation. Each DAG run generates one file per extraction task, following the naming convention `source_YYYYMMDD_HHMMSS.json`.

File Inventory

Table 7 lists the files produced during the representative run of 2026-04-19 at 16:40 UTC.

Table 7: Bronze files generated in the run of 2026-04-19 at 16:40 UTC.

Filename	Source	Records	Size
tmdb_now_playing_20260419_164055.json	TMDB /movie/now_playing	20	~32 KB
tmdb_trending_20260419_164055.json	TMDB /trending/all/day	20	~28 KB
tmdb_popular_20260419_164055.json	TMDB /movie/popular	60	~85 KB
tmdb_top_rated_20260419_164055.json	TMDB /movie/top_rated	60	~82 KB
tmdb_tv_20260419_164055.json	TMDB /tv/top_rated	60	~78 KB
rottentomatoes_theaters_20260419_164055.json	RT /browse/movies	~68	~45 KB
rottentomatoes_reviews_20260419_164055.json	RT /browse/audience-reviews	5×10	~60 KB

JSON Structure

The following excerpt shows the top-level envelope of `tmdb_now_playing_20260419_164055.json`, which is representative of all TMDB Bronze files. The pipeline appends two meta-data fields — `ingested_at` and `source` — to the original API response to support traceability in downstream layers.

```
{
  "dates": {
    "maximum": "2026-04-22",
    "minimum": "2026-03-11"
  },
  "page": 1,
  "total_pages": 281,
  "total_results": 5601,
  "ingested_at": "2026-04-19T16:40:56.551175",
```

```

"source": "tmdb_now_playing",
"results": [
  {
    "id": 1226863,
    "title": "The Super Mario Galaxy Movie",
    "popularity": 780.1539,
    "vote_average": 6.805,
    "vote_count": 506,
    "release_date": "2026-04-01",
    "genre_ids": [10751, 35, 12, 14, 16],
    "overview": "Having thwarted Bowser's previous plot..."
  },
  {
    "id": 687163,
    "title": "Project Hail Mary",
    "popularity": 269.6938,
    "vote_average": 8.224,
    "vote_count": 1534,
    "release_date": "2026-03-15",
    "genre_ids": [878, 12],
    "overview": "Science teacher Ryland Grace wakes up..."
  },
  {
    "id": 840464,
    "title": "Greenland 2: Migration",
    "popularity": 201.7213,
    "vote_average": 6.488,
    "vote_count": 875,
    "release_date": "2026-01-07",
    "genre_ids": [12, 53, 878],
    "overview": "Having found the safety of the Greenland bunker..."
  }
]
}

```

Table 8 presents three representative records extracted from the file, illustrating the key numeric and textual fields that flow into the Silver processing stage.

Table 8: Three sample records from `tmdb_now_playing_20260419_164055.json`.

title	vote_avg	popularity	release_date	genre_ids
The Super Mario Galaxy Movie	6.805	780.15	2026-04-01	[10751, 35, 12, 14, 16]
Project Hail Mary	8.224	269.69	2026-03-15	[878, 12]
Greenland 2: Migration	6.488	201.72	2026-01-07	[12, 53, 878]

6.3 Silver Layer Output

The Silver layer applies cleaning, normalisation, and schema enforcement to the raw Bronze data. Outputs are persisted as Parquet files following the naming convention `topic_YYYYMMDD_HHMMSS.parquet`. The file examined here — `silver_titles_20260419_165651.parquet` — was produced automatically by the `silver_processing` DAG, triggered three minutes after the Bronze run via the `FileSensor`.

File Metadata

Table 9: Metadata of `silver_titles_20260419_165651.parquet`.

Property	Value
Filename	<code>silver_titles_20260419_165651.parquet</code>
Produced at	2026-04-19T16:57:01Z (FileSensor trigger)
Records	132 rows
Columns	20
Format	Apache Parquet (Snappy compression)
Bronze sources	<code>rottentomatoes_at_home_20260411_035846.json</code> , <code>tmdb_now_playing_20260411_035846.json</code> (and others across multiple ingestion runs)

Schema

Table 10: Schema of `silver_titles_20260419_165651.parquet`.

Column	Type	Description
<code>canonical_title</code>	string	Lowercase normalised title used as join key across sources
<code>source_title</code>	string	Original title as received from source
<code>source</code>	string	Source identifier(s), e.g. <i>rottentomatoes</i> , <i>tmdb</i>
<code>source_type</code>	string	Endpoint or catalog type for each contributing source
<code>source_id</code>	float64	TMDB numeric ID (null when only RT data available)
<code>url</code>	string	URL to the RT movie page
<code>poster_url</code>	string	CDN URL of the movie poster image
<code>release_date</code>	string	Raw release date string(s) from source(s)
<code>release_year</code>	string	Extracted 4-digit release year
<code>popularity</code>	float64	TMDB popularity score
<code>vote_average</code>	float64	TMDB user rating on a 0–10 scale
<code>vote_count</code>	float64	Total number of TMDB votes
<code>critics_score</code>	float64	Rotten Tomatoes Tomatometer percentage
<code>audience_score</code>	float64	RT Popcornmeter percentage
<code>overview</code>	string	Plot synopsis from TMDB
<code>genre_ids</code>	string	TMDB genre ID list serialised as string
<code>scraped_at</code>	string	Original extraction timestamp from Bronze
<code>bronze_file_name</code>	string	Source Bronze filename(s) for lineage tracking
<code>bronze_source_type</code>	string	Source type label from Bronze metadata
<code>processed_at</code>	string	UTC timestamp when Silver processing ran

Sample Records

The records below were read directly from the Parquet file and illustrate the cross-source enrichment achieved in the Silver layer. Records matched across both TMDB

and Rotten Tomatoes carry the full set of numeric scores; records from a single source carry `null` in the fields contributed by the absent source.

Table 11: Eight sample records from `silver_titles_20260419_165651.parquet`. Dashes indicate `null` values.

<code>canonical_title</code>	<code>source</code>	<code>year</code>	<code>vote avg</code>	<code>votes</code>	<code>critics %</code>	<code>audience %</code>
crime 101	rt + tmdb	2026	7.034	592	89	85
pizza movie	rt + tmdb	2026	6.580	56	78	66
send help	rt + tmdb	2026	7.021	971	93	87
super mario bros.	rt only	2023	—	—	59	95
s4: bob lazar story	rt + tmdb	2026	8.400	8	—	95
anaconda	rt + tmdb	2026	5.879	970	47	75
the housemaid	rt only	2026	—	—	73	92
good luck, have fun	rt + tmdb	2026	6.909	314	83	85

Data Quality Summary

Table 12: Data quality statistics computed from `silver_titles_20260419_165651.parquet`.

Metric	Value
Total records	132
Cross-source matches	34 (26%) — titles present in both TMDB and RT
<code>vote_average</code> range	0.00 – 8.72
<code>critics_score</code> nulls	80 / 132 (61%) — TMDB-only records lack RT scores
<code>audience_score</code> nulls	96 / 132 (73%) — same cross-source gap
<code>release_date</code> nulls	6 / 132 (5%) — titles without a confirmed date
Unique source combos	3: <i>rottentomatoes</i> , <i>tmdb</i> , <i>rottentomatoes</i> , <i>tmdb</i>
Lineage coverage	100% — <code>bronze_file_name</code> populated for all 132 records

Transformations Applied

The Silver DAG applied the following operations to produce this file from the Bronze JSON inputs:

- **Title normalisation:** `canonical_title` derived by lowercasing `source_title` and stripping punctuation, used as the join key when merging TMDB and RT records.
- **Score conversion:** `critics_score` and `audience_score` converted from percentage strings (e.g. 89%) to `float64` numeric values.
- **Cross-source join:** TMDB and RT records matched on `canonical_title`; fields from both sources merged into a single row where a match exists.
- **Metadata injection:** `bronze_file_name` and `processed_at` added to every record for downstream lineage and reproducibility.
- **Type enforcement:** `release_year` extracted as a separate string column; `popularity`, `vote_average`, and `vote_count` cast to `float64`.
- **Parquet serialisation:** Output written using PyArrow with Snappy compression, readable by Pandas and PySpark for the Gold layer.

6.4 Data Exploration Example

The notebook `notebooks/silver_validation.ipynb` demonstrates how to read and explore the Silver Parquet files produced by the pipeline. It is structured in seven sections that progressively validate the three Silver datasets: `silver_titles`, `silver_rt_reviews`, and `silver_tmdb_reviews`.

The notebook begins by resolving the `datalake_silver/` directory relative to its location in `notebooks/`, using `pathlib.Path` to ensure portability across operating systems. A helper function `latest_parquet` automatically selects the most recent file for each prefix, so the notebook always reads the latest Silver output without requiring manual path updates.

Once the three DataFrames are loaded via `pd.read_parquet` with the `pyarrow` engine, the notebook runs the following validation checks:

- **Shape and columns:** confirms that each dataset contains the expected number of rows and the full column schema defined in `utils/silver_processing.py`.

- **Null analysis:** reports null counts and percentages per column, surfacing expected gaps such as `critics_score` being absent for TMDb-only records and `popularity` being absent for RT-only records.
- **Duplicate check:** verifies that the deduplication logic in the Silver processing utilities produced no duplicate rows, using the same key combinations applied during transformation.
- **Source distribution:** inspects how records are distributed across source combinations (*rottentomatoes*, *tmdb*, and *rottentomatoes*, *tmdb*), and confirms the `review_type` split between critic and audience reviews in `silver_rt_reviews`.
- **Text cleaning validation:** compares `review_text` against `review_text_clean` to confirm that the cleaning pipeline — lowercasing, whitespace normalisation, and HTML entity decoding — was applied consistently across both review sources.
- **Lineage evidence:** verifies that `bronze_file_name`, `bronze_source_type`, and `processed_at` are populated for every row, confirming full traceability from Silver back to the originating Bronze file.
- **Workshop 2 summary:** produces a compact summary table reporting row count, column count, unique sources, average null percentage, and the Parquet filename for each dataset.

The notebook can be executed locally against any Silver output by activating the project virtual environment and running `jupyter notebook` from the repository root. No path modifications are required as long as the standard project structure is maintained.

7 Challenges and Decisions

7.1 Technical Challenges

Silver DAG reprocessing accumulated Bronze files

The most significant technical challenge encountered during implementation was the behaviour of the Silver DAG when triggered after multiple Bronze runs had already accumulated files in `datalake_bronze/`. Because the `FileSensor` only detects the presence of files without distinguishing new from previously processed ones, each

Silver run would attempt to transform the entire Bronze volume, including files that had already been processed in earlier runs. As the number of Bronze files grew across days, this produced duplicate records in the Silver Parquet outputs and progressively increased processing time with each new trigger.

Orchestrating DAGs and utility modules without prior experience

The most pervasive challenge throughout the workshop was implementing a multi-DAG Airflow pipeline with modular utility functions for the first time. Coordinating task dependencies using the TaskFlow API, understanding how XCom serialisation works across tasks, correctly mounting volumes so that files written inside containers are visible on the host filesystem, and structuring the `utils/` package so that Airflow's DAG scanner does not attempt to parse it as a DAG — all of these required significant trial and error given no prior experience with Airflow orchestration.

Pyarrow dependency absent in local environment

When attempting to run the Silver validation notebook locally, the `pyarrow` library was not installed in the local Python environment, resulting in an `ImportError` when calling `pd.read_parquet`. This dependency is present inside the Docker container via the `requirements.txt` but was not installed in the local virtual environment used for notebook execution.

7.2 Solutions Implemented

Processed files tracking via Parquet registry

To prevent the Silver DAG from reprocessing previously handled Bronze files, a tracking registry was introduced. After each successful Silver run, the paths of all processed Bronze files are written to a dedicated Parquet file at `datalake_silver/processed_bronze_files`. On every subsequent Silver run, this registry is read first and used to filter the Bronze file list before classification and transformation. Only files absent from the registry are forwarded to the processing tasks. This ensures that the Silver DAG remains idempotent and that processing time does not grow unboundedly as the Bronze volume accumulates.

Modular utility structure with `__init__.py` isolation

The utility modules were organised under `airflow/dags/utils/` and a `__init__.py` file was added to the directory. Airflow treats directories containing an `__init__.py` as

Python packages rather than DAG folders, preventing the scanner from attempting to parse utility files as DAG definitions. This allowed clean separation between orchestration logic in the DAG files and transformation logic in the utility modules, making each component independently readable and testable.

Dependency installation cell in the notebook

The validation notebook was updated to include a first cell that installs `pyarrow` and `pandas` via `subprocess` if they are not already present in the active environment. This makes the notebook self-contained and executable on any team member's machine without requiring prior environment configuration beyond having Python and Jupyter available.

7.3 Design Decisions

LocalExecutor over CeleryExecutor

The pipeline uses `LocalExecutor` rather than the `CeleryExecutor` configured in the official Airflow Docker Compose file. This decision was driven by the project's single-machine development context: with only one computer available for the pipeline, introducing a Redis broker and Celery workers would add infrastructure complexity without providing any parallelism benefit. The `LocalExecutor` allows the scheduler to execute tasks directly, which is sufficient for the workload of this project and significantly reduces the number of running containers and configuration overhead.

FileSensor trigger over a fixed Silver schedule

The Silver DAG carries no fixed schedule (`schedule_interval=None`) and is triggered exclusively by the `FileSensor` operator when new Bronze files are detected. This decouples the ingestion and transformation stages, allowing the Silver DAG to activate only when there is new data to process rather than on a fixed cadence that might fire with no new input. The sensor is configured with a `poke_interval` of 60 seconds and a `timeout` of 20 minutes, which is appropriate for a daily ingestion schedule.

Parquet registry for processed file tracking

Although a simple text file would have been sufficient to track processed Bronze files, the registry was implemented as a Parquet file to maintain consistency with the rest of the pipeline's storage format and to allow structured querying by date or source

type in future workshop stages. It is acknowledged that this approach introduces a single point of failure: if the registry file is deleted or corrupted, the Silver DAG will reprocess all Bronze files on the next run. A more robust alternative for a production system would be a dedicated metadata database table, but for the academic scope of this project the Parquet registry is a practical and sufficient solution.

Atomic Parquet writes for retry safety

Silver Parquet files are written via a write-to-temporary-file-then-rename pattern (`.tmp` extension followed by `os.replace`). This ensures that if a task fails mid-write, no partially written Parquet file is left on disk that could be read as valid data by downstream processes. Combined with the deterministic `run_stamp` derived from the DAG’s logical date, this makes Silver task retries safe and idempotent.

8 Conclusion

Workshop 2 successfully translated the conceptual pipeline architecture defined in Workshop 1 into a fully operational, containerised data engineering system. The two Airflow DAGs — `bronze_ingestion` and `silver_processing` — have been running continuously for three days, producing timestamped Bronze JSON files from the TMDb API and Rotten Tomatoes scraping, and transforming them into structured Silver Parquet datasets through an automated FileSensor-triggered workflow.

The Bronze layer accumulates raw ingestion outputs from five distinct TMDb endpoints and two Rotten Tomatoes sources per daily run, preserving source integrity and traceability through metadata fields injected at ingestion time. The Silver layer applies normalisation, cross-source merging on canonical title, score conversion, and text cleaning to produce three analytically usable datasets: `silver_titles`, `silver_rt_reviews`, and `silver_tmdb_reviews`. All Silver records carry lineage fields that trace each row back to its originating Bronze file.

The principal challenge of the workshop was practical: implementing a multi-service Airflow orchestration system, modular DAG architecture, and idempotent Silver processing without prior experience with these tools. The solutions developed — the processed files registry, the utility package isolation, and the atomic write pattern — reflect design decisions grounded in the specific constraints of the project rather than generic best practices, and each trade-off has been documented accordingly.

The pipeline is now ready to support Workshop 3, where the Silver Parquet outputs will be loaded with PySpark to produce the Gold layer aggregations and feed

the Plotly Dash dashboard. The timestamp-based file naming convention adopted in this workshop was specifically designed with that requirement in mind, as it allows PySpark to load and distinguish outputs from multiple ingestion dates in a single batch operation.